

CASE STUDY-CO4-AT1

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: MLA0302

COURSE NAME: REINFORCEMENT LEARNING

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

- **Bellman Principle of Optimality**
- **Policy Evaluation and Policy Improvement**
- **Value Iteration vs. Policy Iteration**
- **Industrial applications**

1.1 Bellman Principle of Optimality

The **Bellman Principle of Optimality** states that an optimal decision consists of optimal decisions for the remaining states.

The Bellman Optimality Equation is:

$$V^*(s) = \max_a [R(s,a) + \gamma V^*(s')]$$

Where:

- $V^*(s)$ = optimal value of state
- $R(s,a)$ = immediate reward
- γ = discount factor
- $V^*(s')$ = value of next state

Example

Consider a robot:

Start $\rightarrow$ A $\rightarrow$ Goal

Rewards:

- $\text{Start} \rightarrow A = 2$
- $A \rightarrow \text{Goal} = 5$
- $\gamma = 0.9$

First:

$$V(A) = 5$$

Then:

$$V(\text{Start}) = 2 + 0.9(5) = 6.5$$

Therefore:

Optimal value of Start = 6.5

Simple Python Program

```
gamma = 0.9

V_goal = 0
V_A = 5 + gamma * V_goal
V_start = 2 + gamma * V_A

print("Goal:", V_goal)
print("A:", V_A)
print("Start:", V_start)
```

Output

```
Goal: 0
A: 5.0
Start: 6.5
```

Explanation

The value of the current state depends on the reward received now and the value of the next state. This demonstrates the **Bellman Principle of Optimality**.

1.2 Policy Evaluation

Policy Evaluation calculates how good a given policy is.

The Bellman expectation equation is:

$$V\pi(s)=R(s,\pi(s))+\gamma V\pi(s')$$

Example

A warehouse robot follows:

Warehouse $\rightarrow$ Storage $\rightarrow$ Delivery

Suppose the policy is already fixed. Policy Evaluation calculates the expected reward obtained by following this route.

For example:

$$V(\text{Storage})=5 \quad V(\text{Warehouse})=2+0.9(5)=6.5$$

Therefore, the policy value at Warehouse is **6.5**.

Purpose: To answer "**How good is my current policy?**"

1.3 Policy Improvement

Policy Improvement finds a better action using the values calculated during Policy Evaluation.

Example

A robot has three possible actions:

Action Value

Left 6

Right 9

Up 5

The best action is:

$$\max(6,9,5)=9$$

Therefore:

New policy = Move Right

Purpose: To answer "Can I improve my current policy?"

Policy Evaluation and Policy Improvement are repeated in **Policy Iteration**.

1.4 Value Iteration

Value Iteration directly updates the value of every state using the Bellman Optimality Equation:

$$V_{k+1}(s) = \max_a [R(s,a) + \gamma V_k(s')]$$

Example

A robot can choose:

Left → Reward = 4

Right → Reward = 10

The algorithm eventually selects **Right** because it gives the higher expected value.

Process

Initialize Values



Update Values



Choose Best Action



Repeat



Optimal Policy

1.5 Policy Iteration

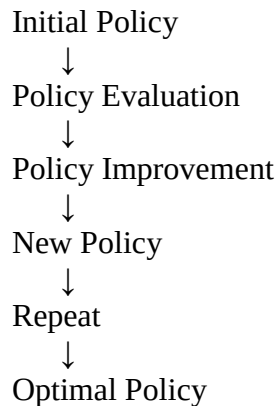
Policy Iteration has two main steps:

Step 1: Policy Evaluation

Calculate the value of the current policy.

Step 2: Policy Improvement

Select better actions based on the calculated values.



1.6 Value Iteration vs Policy Iteration

Feature	Value Iteration	Policy Iteration
Main operation	Value update	Evaluation + Improvement
Policy	Extracted after convergence	Improved every iteration
Calculation	Direct maximization	Two-step process
Goal	Optimal value and policy	Optimal policy

Simple Difference

Value Iteration: Find the best **value** first.

Policy Iteration: Evaluate and improve the **policy** repeatedly.

1.7 Industrial Applications

Warehouse Robots

Find optimal routes while reducing travel time and energy.

UAV/Drone Navigation

Find safe paths while considering obstacles and battery usage.

Autonomous Vehicles

Select optimal speed, lane and route based on traffic conditions.

Traffic Management

Optimize traffic signal timings to reduce congestion.

Manufacturing

Optimize machine scheduling, resource allocation and production.

2. Monte Carlo Methods and Temporal-Difference Learning: A Comparative Study

- **Monte Carlo Prediction and Control**
- **TD(0), SARSA, and Q-Learning**
- **Sample efficiency**
- **Performance comparison through experiments**

2.1 Monte Carlo Prediction

Monte Carlo Prediction estimates the value of a state using the **complete return from an episode**.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Example

Suppose state A gives returns:

Episode 1 = 8

Episode 2 = 6

Episode 3 = 10

Then:

$$V(A) = \frac{8 + 6 + 10}{3} = 8$$

Therefore:

Value of A = 8

Key Point

Monte Carlo waits until the **episode ends** before updating.

2.2 Monte Carlo Control

Monte Carlo Control learns the best action using Q-values.

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

Example

Action Q-value

Left 5

Right 9

Up 6

The agent chooses:

Right, because its Q-value is 9.

2.3 TD(0)

TD(0) learns after every step and does not need to wait for the episode to finish.

$$V(s) \leftarrow V(s) + \alpha [R + \gamma V(s') - V(s)]$$

Example

Given:

- $V(s) = 5$
- $\text{Reward} = 2$
- $V(s') = 8$

- $\alpha=0.1$
- $\gamma=0.9$

$$V(s)=5+0.1[2+0.9(8)-5] \quad V(s)=5.42$$

Therefore, the updated value is **5.42**.

2.4 SARSA

SARSA is an **on-policy** Temporal-Difference algorithm.

SARSA means:

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ State $\rightarrow$ Action

Formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma Q(s',a') - Q(s,a)]$$

It uses the **actual next action selected by the agent**.

Example

```

State A
  ↓
Action Right
  ↓
Reward
  ↓
State B
  ↓
Action Up

```

SARSA uses the Q-value of the **Up** action in State B.

2.5 Q-Learning

Q-Learning is an **off-policy** Temporal-Difference algorithm.

Formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

It uses the **maximum Q-value of the next state**.

Example

At the next state:

Action Q-value

Left 4

Right 8

Up 6

Q-Learning uses:

$$\max(4, 8, 6) = 8$$

Therefore, it uses **8** in its update.

2.6 Sample Efficiency

Sample efficiency means how effectively an algorithm learns from available experience.

Monte Carlo

Complete Episode



Calculate Return



Update

It must wait until the episode ends.

TD Learning

State → Action → Reward



Update

It learns after every step.

Therefore, TD methods can learn continuously from experience.

2.7 Performance Comparison Through Experiment

The following small experiment compares **SARSA** and **Q-Learning**.

```
import numpy as np
import random

Q_sarsa = np.zeros((4, 2))
Q_qlern = np.zeros((4, 2))

alpha = 0.1
gamma = 0.9

for episode in range(100):
    s = 0

    while s != 3:
        a = random.randint(0, 1)
        ns = min(3, s + a + 1)
        r = 10 if ns == 3 else -1

        # SARSA
        na = random.randint(0, 1)
        Q_sarsa[s,a] += alpha * (
            r + gamma * Q_sarsa[ns,na] - Q_sarsa[s,a]
        )

        # Q-Learning
        Q_qlern[s,a] += alpha * (
            r + gamma * np.max(Q_qlern[ns]) - Q_qlern[s,a]
        )

    s = ns

print("SARSA Q-Table:")
print(Q_sarsa)

print("\nQ-Learning Q-Table:")
print(Q_qlern)
```

Experiment Observation

After training:

- SARSA learns using the **actual next action**.
 - Q-Learning learns using the **best possible next action**.
 - Q-values increase as the agents gain experience.
 - The learned Q-tables can be compared to evaluate performance.
-